

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)**

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и облачные технологии**

Направление подготовки **09.03.03 Прикладная информатика**

**Отчет
по практической работе №1
«Решение задач линейного программирования» по
дисциплине «Методы оптимизации »**

Выполнили: Субагио Сатрио поток 1.3 К3340
Преподаватель : Духанов Алексей Валентинович

Дата 07.10.2026

Санкт-Петербург 2026

1. Постановка задачи

8.

Минимизировать $Z = x_1 + 3x_2 + 2x_3 + x_4$

при условиях:

$$x_1 + x_2 + 2x_4 \leq 8$$

$$x_2 + x_3 + x_4 = 6$$

$$2x_1 + x_3 \geq 2$$

2. Решение симплекс-методом

1. Добавление дополнительных переменных (искусственных переменных) приводит ограничения к виду равенства

$$\begin{cases} x_1 + x_2 + 2x_4 + x_5 = 8 \\ x_2 + x_3 + x_4 = 6 \\ 2x_1 + x_3 - x_6 = 2 \\ x_i \geq 0 \end{cases} \quad x_5, x_6 \geq 0$$

2. Добавив искусственные переменные

$$x_5 + x_7 + x_2 + 2x_4 = 8$$

$$x_7 + x_2 + x_3 + x_4 = 6$$

$$x_2 + 2x_1 + x_3 - x_6$$

3. Вспомогательная задача

$$W(x_0) = x_7 + x_2 \rightarrow \min$$

$$x_7 = 6 - x_2 - x_3 - x_4$$

$$x_2 = 2 - 2x_1 - x_3 + x_6$$

$$W = (6 - x_2 - x_3 - x_4) + (2 - 2x_1 - x_3 + x_6)$$

$$W = 8 - 2x_1 - x_2 - 2x_3 - x_4 + x_6 = 8 \rightarrow Q = 8$$

4. Начальная симплекс-таблица

	x_1	x_2	x_3	x_4	x_5	b
x_5	1	1	0	2	0	8
y_1	0	1	1	1	0	6
y_2	②	0	1	0	-1	2
w	-2	-1	-2	-1	7	8

Самый отрицательный

коэффициент: -2 $\rightarrow$ ②

ключевой элемент: 2 $\rightarrow$ ①
pivot

	x_1	x_3	x_4	x_5	b
x_5	1	-0,5	2	0,5	7
y_1	①	1	1	0	6
x_1	0	0,5	0,5	-0,5	1
w	-1	-1	-1	0	6

	x_3	x_4	x_5	b
x_5	-1,5	1	0,5	7
x_2	1	1	0	6
x_1	0,5	0	-0,5	1
w	0	0	0	0

Свободные $x_3 = x_4 = x_5 = 0$

Базис $x_5 = 1$ $x_2 = 6$ $x_1 = 1$

5. проверка

$$x_1 + x_2 + 2x_4 \leq 8 \quad 1 + 6 + 2 \cdot 0 \leq 8$$

$$x_2 + x_3 + x_4 = 6 \Leftrightarrow 6 + 0 + 0 = 6$$

$$2x_1 + x_3 \geq 2 \quad 2 \cdot 1 + 0 \geq 2$$

6. из таблицы выразим z через независимые переменные

$$x_1 = 1 - 0,5x_3 + 0,5x_5, x_2 = 6 - x_3 - x_4, x_5 = 1 + 1,5x_3 - x_4 - 0,5x_6$$

$$Z = x_1 + 3x_2 + 2x_3 + x_4$$

$$Z = (1 - 0,5x_3 + 0,5x_6) + 3(6 - x_3 - x_4) + 2x_3 + x_4$$

$$Z = 19 - 1,5x_3 - 2x_4 + 0,5x_6$$

47. Находим С.Т

	x_3	x_4	x_6	b
x_5	-1,5	1	0,5	7
x_2	1	0	0	6
x_1	0,5	0	-0,5	1
Z	-1,5	-2	0,5	19

x_4	x_3	x_5	x_6	b		x_4	x_5	x_6	b
-1,5	1	0,5	7		$\rightarrow$	3	1	-1	4
2,5	-1	-0,5	5			-5	-1	2	0
0,5	0	-0,5	1			2	0	-1	2
-1,5	2	1,5	17			9	2	-3	3

x_4	x_3	x_5	b
0,5	0,5	0,5	4
-0,5	0,5	-0,5	2
-2,5	0,5	-0,5	0
1,5	1,5	0,5	3

базис : (x_4, x_3, x_6)

$x_1 = x_2 = x_5 = 0 \rightarrow x_4 = 4, x_3 = 2, x_6 = 0$

Итого получаем решение $(x_1, x_2, x_3, x_4) = (0, 0, 2, 4)$

$Z^* = 8$

проверка исходных ~~данных~~ условий

$$\begin{cases} x_1 + x_2 + 2x_4 \leq 8 \\ x_2 + x_3 + x_4 = 6 \\ 2x_1 + x_3 \geq 2 \end{cases} \Leftrightarrow \begin{cases} 0 + 0 + 2 \cdot 4 \leq 8 \\ 0 + 2 + 4 = 6 \\ 2 \cdot 0 + 2 \geq 2 \end{cases}$$

3. Решение задачи в Excel Solver

Для проверки полученного результата задача была дополнительно решена в Microsoft Excel с помощью надстройки Solver.

В качестве изменяемых ячеек были заданы переменные: x_1, x_2, x_3, x_4 (диапазон $\$C\$6:\$F\6).

Целевая функция: $Z = x_1 + 3x_2 + 2x_3 + x_4$ была задана на минимизацию (ячейка $\$G\5).

Задача линейного программирования №8 — минимизация							
$Z = x_1 + 3x_2 + 2x_3 + x_4 \rightarrow \min$							
	x1	x2	x3	x4	Целевая функция (min)		
с	1	3	2	1	Z =	8.	
х	0	0	2	4			
Ограничения:							
	x1	x2	x3	x4	Знак	RHS	Левая часть
Огр. 1	1	1	0	2	<=	8	8
Огр. 2	0	1	1	1	=	6	6
Огр. 3	2	0	1	0	>=	2	2

Рис 1 – Заданные параметры и целевая функция (до запуска Solver).

В Solver были добавлены следующие ограничения:

$$x_1 + x_2 + 2x_4 \leq 8$$

$$x_2 + x_3 + x_4 = 6$$

$$2x_1 + x_3 \geq 2$$

$$x_i \geq 0$$

Для решения был выбран метод Simplex LP, поскольку рассматриваемая задача является задачей линейного программирования.

\$G\$5

To:

☐ Max ☒ Min ☐ Value Of:

By Changing Variable Cells:

\$C\$6:\$F\$6

Subject to the Constraints:

\$I\$10 <= \$H\$10

\$I\$11 = \$H\$11

\$I\$12 >= \$H\$12

\$C\$6:\$F\$6 >= 0

Add Change Delete

Рис 2 – Solver Parameters в Excel.

После выполнения расчёта Solver получил следующие значения:

$$x_1 = 0, x_2 = 0, x_3 = 2, x_4 = 4$$

Значение целевой функции: $Z^* = 8$

Результат, полученный с помощью Excel Solver, полностью совпадает с результатом, найденным вручную симплекс-методом. Следовательно, ручное решение симплекс-методом выполнено корректно, а результат подтверждается вычислением в Excel Solver.

4. Решение задачи программным способом

Исходный код реализует симплекс-метод в два этапа (вспомогательная и основная задачи), без использования готовых библиотек для симплекс-метода. Каждый этап решения вынесен в отдельную функцию: чтение входных данных, приведение к каноническому виду, построение вспомогательной и основной целевой функции, выбор разрешающего элемента, пересчёт таблицы, извлечение решения.

Пример входных данных (input.txt):

```
min
4
1 3 2 1
3
1 1 0 2 <= 8
0 1 1 1 = 6
2 0 1 0 >= 2
```

Результат выполнения:

```
PS C:\Users\MSI\Metopt26\Lab1> py simplex.py

=====
РЕЗУЛЬТАТ:
Вектор x: [0.0, 0.0, 2.0, 4.0]
Оптимальное значение F: 8.0
=====
```

ИСХОДНЫЙ КОД:

```
def read_file(file_path: str) -> tuple:
    with open(file_path, 'r', encoding='UTF8') as file:
        task_type = file.readline().strip()
        n = int(file.readline().strip())
        if n <= 0:
            raise ValueError("invalid data: n must be positive")
        c = file.readline().split()
        if len(c) < n:
            raise ValueError("invalid data: недостаточно коэффициентов c")
        if task_type not in ('max', 'min'):
            raise ValueError("invalid type: task_type должен быть 'max' или 'min'")
        m = int(file.readline().strip())
        if m <= 0:
            raise ValueError("invalid data: m must be positive")
        data = file.readlines()
        if len(data) < m:
            raise ValueError("invalid data: недостаточно строк ограничений")
    return n, c, m, data, task_type

def to_canonical(n, c, m, data, task_type) -> tuple:
    if task_type == 'max':
        c = [-1.0 * float(k) for k in c]
    else:
        c = [float(k) for k in c]
    slack_count = sum(1 for row in data if row.strip().split()[n] in ('<=', '>='))
    new_n = n + slack_count
    parsed = [[0.0] * (new_n + 1) for _ in range(m)]
    k = 0
    for i in range(m):
        list_row = data[i].strip().split()
        parsed[i][:n] = map(float, list_row[:n])
        sign = list_row[n]
        if sign == '>=':
            parsed[i][n + k] = -1.0
            k += 1
        elif sign == '<=':
            parsed[i][n + k] = 1.0
            k += 1
        parsed[i][-1] = float(list_row[-1])
        if parsed[i][-1] < 0:
            parsed[i] = [-x for x in parsed[i]]
    return new_n, c, m, parsed

def extend_objective_coefficients(c, n, m) -> list:
    return list(c) + [0.0] * (n + m - len(c))

def initial_basis_and_free(n, m) -> tuple:
    basis = [n + i for i in range(m)]
    free = [j for j in range(n)]
    return basis, free

def build_auxiliary_objective(n, m, a) -> list:
    row = [0.0] * (n + 1)
    row[:-1] = [-sum(a[i][j] for i in range(m)) for j in range(n)]
    row[-1] = -sum(a[i][-1] for i in range(m))
    return row

def build_main_objective(n, m, c, a, basis, free) -> list:
    row = [0.0] * (n + 1)
    for j in range(n):
```

```

        value = sum(c[basis[i]] * a[i][j] for i in range(m))
        row[j] = -(value - c[free[j]])
    row[-1] = -sum(c[basis[i]] * a[i][-1] for i in range(m))
    return row
def choose_pivot(n, m, table) -> tuple:
    reduced_costs = table[-1][:-1]
    rhs = [table[i][-1] for i in range(m)]
    min_cost = min(reduced_costs)
    if min_cost >= -1e-9:
        return -1, -1
    pivot_col = reduced_costs.index(min_cost)
    pivot_row = -1
    for i in range(m):
        if table[i][pivot_col] > 1e-9 and rhs[i] >= 0:
            if pivot_row == -1:
                pivot_row = i
            elif rhs[i] / table[i][pivot_col] < rhs[pivot_row] /
table[pivot_row][pivot_col]:
                pivot_row = i
    if pivot_row == -1:
        return -1, -1
    return pivot_row, pivot_col
def pivot_table(n, m, table, pivot_row, pivot_col, is_auxiliary) -> list:
    pivot_value = table[pivot_row][pivot_col]
    new_table = [[0.0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        for j in range(n + 1):
            if i == pivot_row and j == pivot_col:
                new_table[i][j] = 0.0 if is_auxiliary else 1.0 / pivot_value
            elif j == pivot_col:
                new_table[i][j] = 0.0 if is_auxiliary else -table[i][j] /
pivot_value
            elif i == pivot_row:
                new_table[i][j] = table[i][j] / pivot_value
            else:
                new_table[i][j] = table[i][j] - table[pivot_row][j] *
table[i][pivot_col] / pivot_value
    return new_table
def run_phase(n, m, table, basis, free, is_auxiliary) -> tuple:
    pivot_row, pivot_col = choose_pivot(n, m, table)
    if pivot_row == -1:
        return table, basis, free
    table = pivot_table(n, m, table, pivot_row, pivot_col, is_auxiliary)
    basis[pivot_row], free[pivot_col] = free[pivot_col], basis[pivot_row]
    return run_phase(n, m, table, basis, free, is_auxiliary)
def solve_auxiliary_problem(n, m, a, basis, free) -> tuple:
    objective_row = build_auxiliary_objective(n, m, a)
    table = a + [objective_row]
    table, basis, free = run_phase(n, m, table, basis, free, is_auxiliary=True)
    EPS = 1e-6
    if abs(table[-1][-1]) > EPS:
        raise ValueError("допустимое решение не найдено: задача несовместна")
    return table[:-1], basis, free
def solve_main_problem(n, m, c, a, basis, free) -> tuple:
    objective_row = build_main_objective(n, m, c, a, basis, free)
    table = a + [objective_row]
    return run_phase(n, m, table, basis, free, is_auxiliary=False)
def extract_solution(n_orig, table, basis, task_type) -> tuple:
    x = [0.0] * n_orig
    for i, var in enumerate(basis):
        if var < n_orig:

```

```

        x[var] = table[i][-1]
    f_val = table[-1][-1]
    if task_type == 'min':
        f_val = -f_val
    EPS_ROUND = 9
    x = [round(v, EPS_ROUND) for v in x]
    f_val = round(f_val, EPS_ROUND)
    return x, f_val
def print_solution(x, f_val) -> None:
    print("\n" + "=" * 30)
    print("РЕЗУЛЬТАТ:")
    print(f"Вектор x: {x}")
    print(f"Оптимальное значение F: {f_val}")
    print("=" * 30)
def main() -> None:
    n, c, m, data, task_type = read_file('input.txt')
    n_canon, c_raw, m, a = to_canonical(n, c, m, data, task_type)
    c_canon = extend_objective_coefficients(c_raw, n_canon, m)
    basis, free = initial_basis_and_free(n_canon, m)
    a, basis, free = solve_auxiliary_problem(n_canon, m, a, basis, free)
    final_table, basis, free = solve_main_problem(n_canon, m, c_canon, a, basis,
free)
    x, f_val = extract_solution(n, final_table, basis, task_type)
    print_solution(x, f_val)
if __name__ == "__main__":
    main()

```

Результат, полученный вручную, с помощью Excel Solver и с помощью программы на Python, совпадает: $x = (0, 0, 2, 4)$, $Z(x) = 8$.

Исходный код программы доступен по ссылке:

<https://github.com/TioSatrio100/Metopt26/tree/main/Lab1>

Рефлексивный вывод

Полезным в ходе выполнения данной работы оказалось понимание того, как привести задачу линейного программирования к каноническому виду и применить симплекс-метод для получения оптимального результата.

Основная трудность заключалась в том, что при первом знакомстве с темой материал понимался не сразу, из-за чего пришлось дополнительно искать объяснения в других источниках, в том числе на английском и индонезийском языках. Дополнительную сложность создавало то, что работа выполнялась после каникул, в связи с чем потребовалось заново «разогреть» навыки логического и математического мышления. Эти трудности были преодолены за счёт пошагового разбора каждого этапа решения и последующей проверки результата независимыми способами (Excel Solver и программа на Python).

